26-2 유니티 스터디

2 주차

정혜원

오늘 만드는 것 & 진행 순서

프리팹으로 다양한 레벨의 Dongle 생성하기

① GameManager 만들기 → ② Dongle 생성(Instantiate) → ③ 코루틴으로 반복

④ 레벨 & 애니메이션 → ⑤ 랜덤 레벨 → ⑥ 프레임 · 물리 보정

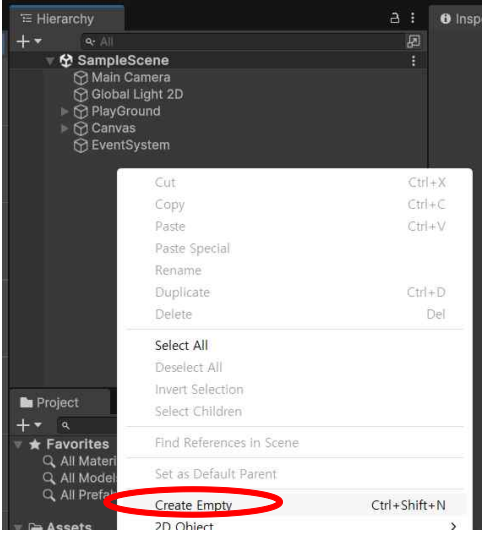
GameManager 만들기

Hierarchy → Create Empty → 이름 'GameManager', 위치 (0, 0)

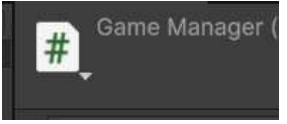
Scripts 폴더에 같은 이름의 C# Script 생성

단, 스크립트 이름에는 공백이 들어가면 안 돼요!

오브젝트에 스크립트를 드래그해서 연결 (매니저는 Hierarchy 맨 위에 두면 편해요)



GameManager 만들기

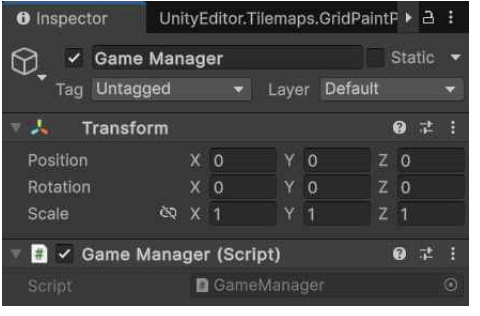


Hierarchy → Create Empty → 이름 'GameManager', 위치 (0, 0, 0)

Scripts 폴더에 같은 이름의 C# Script 생성

단, 스크립트 이름에는 공백이 들어가면 안 돼요!

오브젝트에 스크립트를 드래그해서 연결 (매니저는 Hierarchy 맨 위에 두면 편해요)



lastDongle 변수 만들기

```
public class GameManager : MonoBehaviour
{
    public Dongle lastDongle;
    // Start is called once before the fi
    void Start()
    {

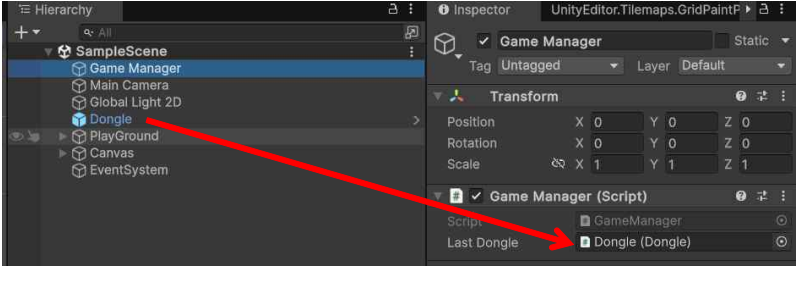
    }
```

씬에 올라와 있는 Dongle을 보관할 변수가 필요해요

public Dongle lastDongle; 선언

public 덕분에 Inspector 창에 변수가 공개됩니다

씬에 있는 Dongle을 드래그해서 변수 안에 넣어주기!



TouchDown() / TouchUp() 함수

지난 시간 Touch Pad → Dongle로 바로 연결했던 걸 GameManager가 대신 해줄 거예요

TouchDown(): lastDongle.Drag() 호출

TouchUp(): lastDongle.Drop() 호출

```
// Update is called once per frame
void Update()
{

}

public void TouchDown()
{
    lastDongle.Drag();
}
public void TouchUp()
{
    lastDongle.Drop();
}
```

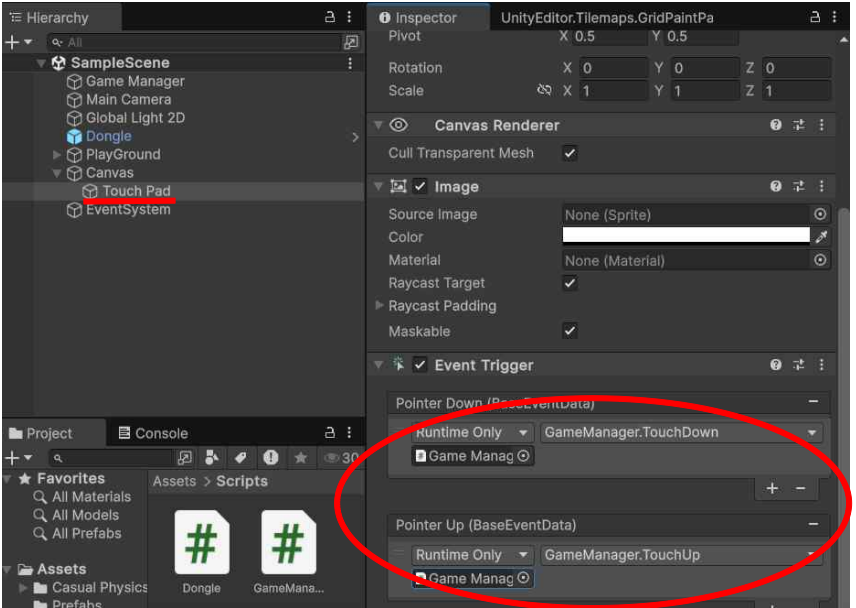
Event Trigger 연결 바꾸기

Touch Pad의 Event Trigger 연결 대상을 Dongle → GameManager로 변경

Pointer Down → GameManager.TouchDown()

Pointer Up → GameManager.TouchUp()

겉보기엔 지난 시간과 똑같지만, 이제 로직이 GameManager에서 시작돼요!


Hierarchy
All
SampleScene
Game Manager
Main Camera
Global Light 2D
Dongle
PlayGround
Canvas
Touch Pad
EventSystem
Inspector
UnityEditor.Tilemaps.GridPaintPa
Pivot
X 0.5
Y 0.5
Rotation
X 0
Y 0
Z 0
Scale
X 1
Y 1
Z 1
Canvas Renderer
Cull Transparent Mesh
Image
Source Image
None (Sprite)
Color
Material
None (Material)
Raycast Target
Raycast Padding
Maskable
Event Trigger
Pointer Down (BaseEventData)
Runtime Only
GameManager.TouchDown
Game Manag
Pointer Up (BaseEventData)
Runtime Only
GameManager.TouchUp
Game Manag
Project
Console
Favorites
All Materials
All Models
All Prefabs
Assets
Casual Physics
Prefabs
Assets > Scripts
Dongle
GameMana...

Dongle 생성 함수 준비

오늘의 하이라이트: Dongle 생성! (Update()는 필요 없으니 지워주세요)

NextDongle(): 다음 Dongle을 가져오는 함수

GetDongle(): Dongle을 실제로 생성하는 함수

생성에는 지난 시간에 만든 Dongle Prefab을 사용합니다

```
public class GameManager : MonoBehaviour
{
    public Dongle lastDongle;
    public GameObject donglePrefab;

    void Start()
    {

    }

    void GetDongle()
    {

    }

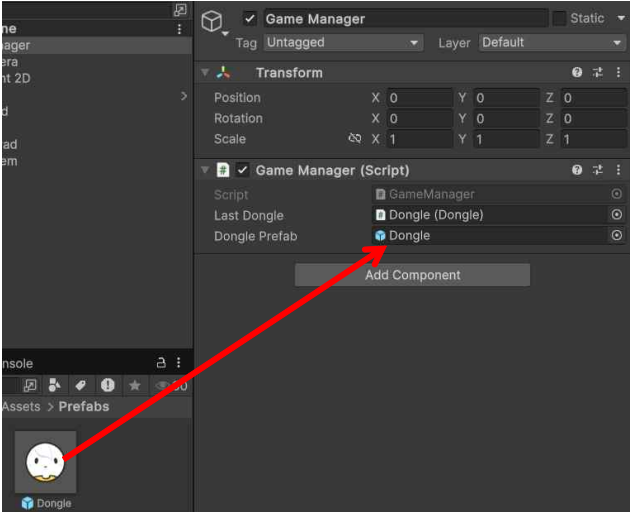
    void NextDongle()
    {

    }

    public void TouchDown()
    {
```

Prefab 변수 & Instantiate()

public GameObject donglePrefab; 선언 → Inspector에서 Prefab 드래그해서 초기화



Prefab 변수 & Instantiate()

GetDongle()에서 Instantiate(donglePrefab) → 호출할 때마다 클론이 생성돼요

```
void GetDongle()
{
    Instantiate(donglePrefab);
}
```

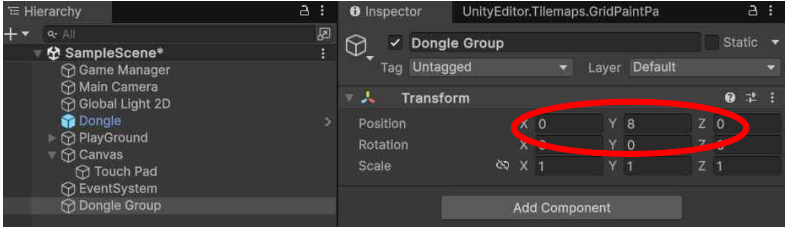
Dongle Group 만들기

Create Empty → 'Dongle Group' 생성,
Playground 바로 아래에 배치

위치는 Y 8 → 앞으로 Dongle이 생성될 높이

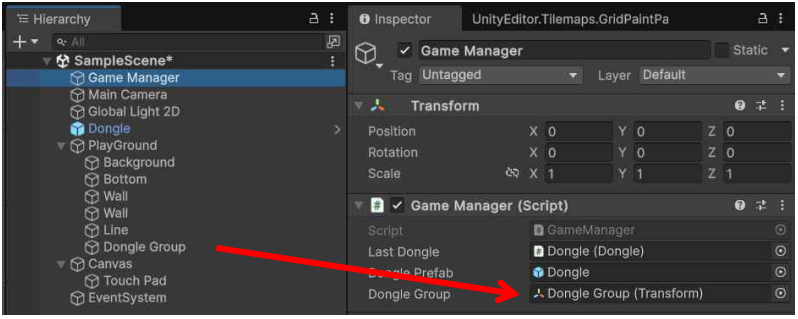
public Transform dongleGroup; 선언

```
public class GameManager : MonoBehaviour
{
    public Dongle lastDongle;
    public GameObject donglePrefab;
    public Transform dongleGroup;
```



Dongle Group 만들기

Inspector에서 Dongle Group을 드래그해서 초기화



부모를 지정해서 생성하기

Instantiate(donglePrefab, dongleGroup) → 두 번째 인자 = 부모

생성된 Dongle은 Dongle Group의 자식으로 들어가고

위치도 부모 기준이라 자동으로 Y 8에서 생성!

```
Dongle GetDongle()
{
    return Instantiate(donglePrefab, dongleGroup);
}

void NextDongle()
{
    GetDongle();
}
```

GetDongle()에 반환값 주기

생성한 Dongle을 변수에 담아 조작하고 싶어요 → 반환형을 void → Dongle로

Dongle newDongle = GetDongle(); → 따끈따끈한 새 Dongle 받기

Start()에서 NextDongle() 호출

GetDongle()에 반환값 주기

```
void Start()
{
    NextDongle();
}

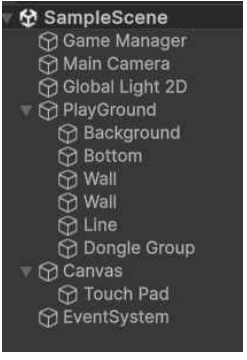
Dongle GetDongle()
{
    GameObject instant = Instantiate(donglePrefab, dongleGroup);
    Dongle instantDongle = instant.GetComponent<Dongle>();
    return instantDongle;
}

void NextDongle()
{
    Dongle newDongle = GetDongle();
    lastDongle = newDongle;
}
```

NextDongle() 완성 & 첫 실행

씬에 원래 있던 Dongle은 **Delete**로 삭제

Save & Save Project 후 실행 → Dongle Group 아래에 **클론 생성!**

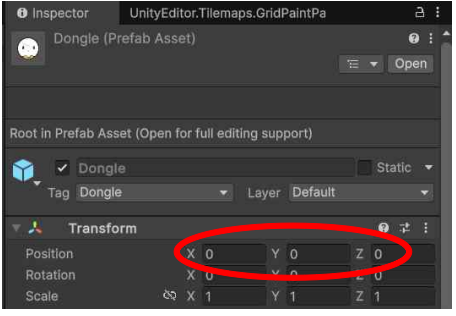


버그 ①: Dongle이 너무 위에 생겨요

생성 위치가 Y 8이 아니라 훨씬 위로?!

원인: Prefab의 Position Y에 8이 남아 있음 → 부모 8 + 8

Prefab을 만들 때 Position은 (0, 0, 0)으로 초기화하기!



버그 ②: 클릭하면 에러가 나요

원인: 씬의 Dongle을 지워서 lastDongle이 비어 있음

NextDongle()에서 lastDongle = newDongle; 로 저장

그래도 Dongle이 나오기 전에 클릭하면 또 에러가 나요!

TouchDown / TouchUp 보정하기

if (lastDongle == null) return; → 비어 있으면 함수 탈출

void 함수도 return 가능! 그 아래 로직은 실행되지 않아요

TouchUp()에도 똑같이 추가

Drop() 후 lastDongle = null; → 손을 뗀 Dongle은 더 이상 관리 X

```
public void TouchDown()
{
    if (lastDongle == null)
        return;
    lastDongle.Drag();
}
public void TouchUp()
{
    if (lastDongle == null)
        return;
    lastDongle.Drop();
    lastDongle = null;
}
```

여러 개 만들기: 무한 루프 주의!

NextDongle()이 자기 자신을 다시 호출하면? → 무한 반복으로 Unity가 멈춰요

lastDongle이 비워질 때까지 기다리는 장치가 필요

그래서 코루틴(Coroutine)을 사용합니다!

코루틴(Coroutine)이란?

진행 순서를 Unity에게 맡기는 특별한 함수

IEnumerator WaitNext() 로 선언 (IEnumerator = 열거형 인터페이스)

코루틴에는 yield 키워드가 반드시 필요!

yield return null → 한 프레임 쉬기 / yield return new WaitForSeconds(2.5f) → 2.5초 쉬기

WaitNext() 완성

while (lastDongle != null) → **Dongle을 떨어뜨릴 때까지 반복하며 대기**

while 안에 yield return null; 이 없으면 한 프레임에 무한 반복 → Unity 멈춤!

대기가 끝나면 2.5초 쉬고 → NextDongle() 호출

코루틴 호출: StartCoroutine()

WaitNext() 처럼 일반 함수처럼 호출하면 안 돼요!

StartCoroutine(WaitNext()) 또는 StartCoroutine("WaitNext")

NextDongle() 마지막에 호출 → 떨어뜨리고 2.5초 뒤 다음 Dongle 등장!

```
void NextDongle()
{
    Dongle newDongle = GetDongle();
    lastDongle = newDongle;

    StartCoroutine(WaitNext());
}

IEnumerator WaitNext()
{
    while (lastDongle != null)
    {
        yield return null;
    }

    yield return new WaitForSeconds(2.5f);
    NextDongle();
}
```

```
1  using System.Collections;
2  using UnityEngine;
3
```

Dongle에 레벨 주기

지금은 레벨 0인 Dongle 0만 계속 나와요

Dongle 스크립트에 public int level; 추가

레벨은 0 ~ 7, 총 8가지 Dongle

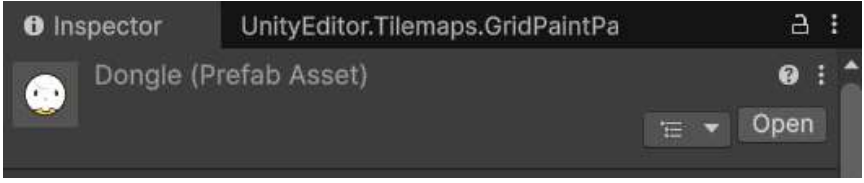
숫자에 따라 스프라이트를 바꾸는 건 애니메이션으로 해볼게요!

```
public class Dongle : MonoBehaviour
{
    public int level;
    public bool isDrag;
    Rigidbody2D rigid;
```

Animator 추가하기

Animation 폴더에 Animator와 Level 2~7은 제공, Level 0 · 1은 직접 제작!

Prefab 더블클릭 (또는 Open Prefab) → Prefab 모드



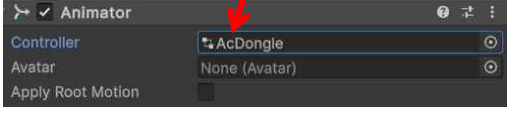
Animator 추가하기

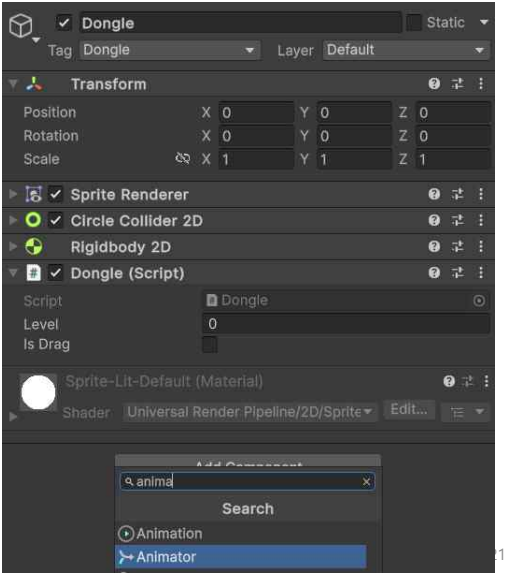
Add Component → Animator, Controller에 'AcDongle' 지정

Controller 더블클릭 → Animator 창

(안 열리면 Window → Animation → Animator)







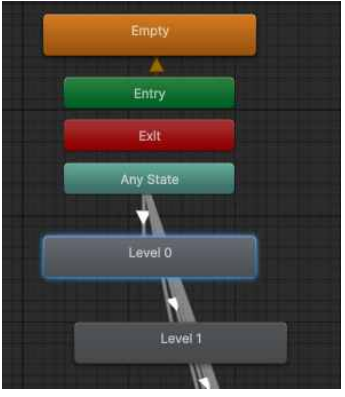
Animator 창 살펴보기

AcDongle 컨트롤러 더블클릭

Level 0 ~ 7 상태와 트랜지션(화살표)이 이미 연결되어 있어요

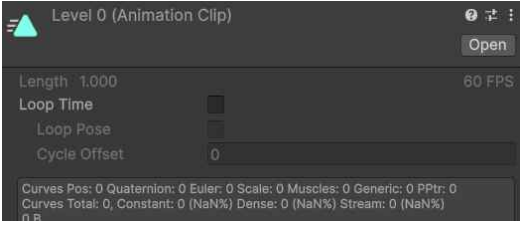
화살표 클릭 → 'Level' 파라미터 값에 따라 해당 상태로 이동

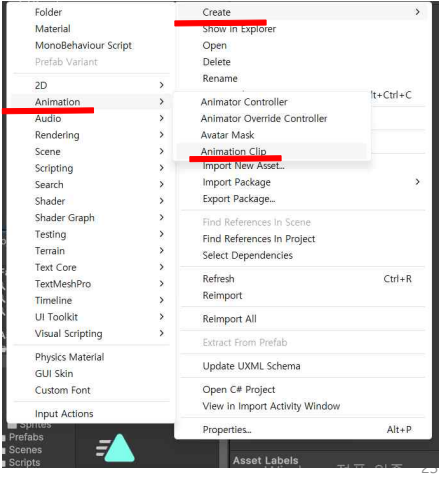
단, Level 0 · 1은 애니메이션(Motion)이 비어 있어요



Level 0 애니메이션 만들기

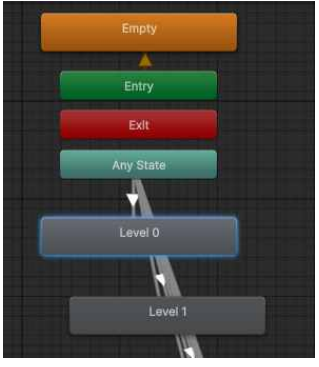
Animation 폴더 우클릭 → Create → Animation → 'Level 0'



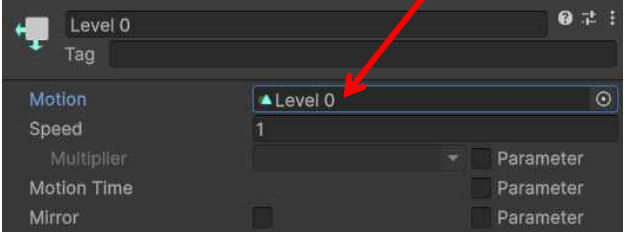


Level 0 애니메이션 만들기

Animator에서 Level 0 상태 선택 → Motion에 Level 0 연결



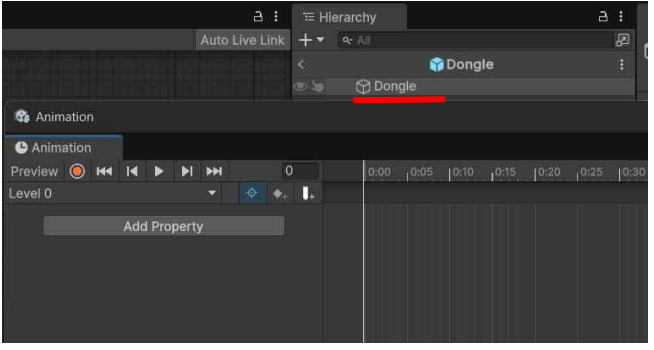




Level 0 애니메이션 만들기

Level 0 애니메이션 더블클릭으로 Animation 창 열기 →

Prefab을 선택하면 Add Property 활성화!



애니메이션이란?

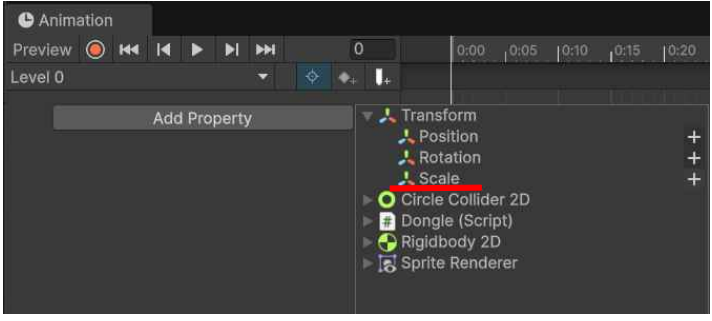
애니메이션은 그림만 바꾸는 게 아니라

오브젝트에 포함된 컴포넌트들을 시간대별로 바꾸는 것!

물리 · 스프라이트 · 트랜스폼 전부 가능 → 오늘은 **Transform + Sprite Renderer**

Scale 키프레임 넣기

Add Property → Transform → Scale

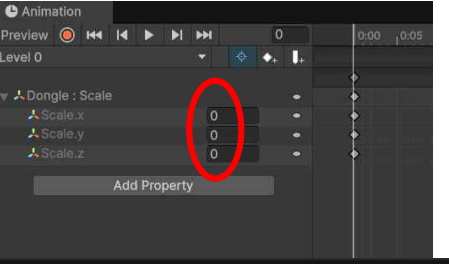


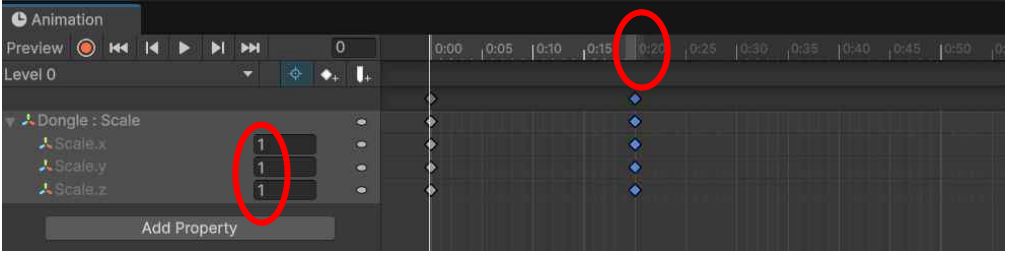
Scale 키프레임 넣기

0초: Scale 모두 0 (처음엔 안 보이다가)

마지막 키프레임을 0.2초로 옮기고 Scale 1

미리보기로 Scene에서 커지는 모습 바로 확인!

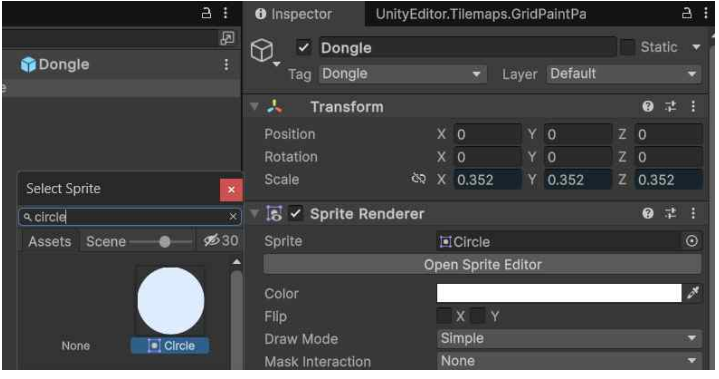




Sprite 바꾸기

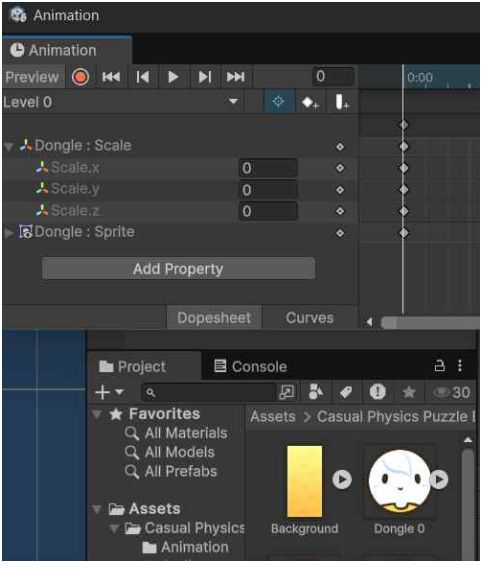
Add Property → Sprite Renderer → Sprite

테스트용: Sprite 선택 창에서 숨겨진 기본 에셋 표시 → 'Circle'로 바꿔두기



Sprite 바꾸기

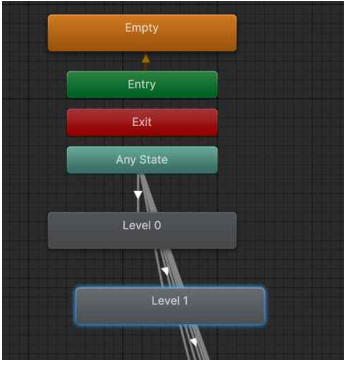
원하는 스프라이트를 키프레임에 드래그 하면 끝! (Level 0 → Dongle 0)



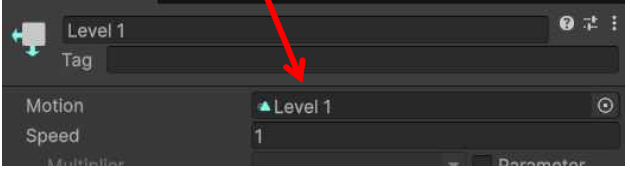
Level 1 애니메이션

Level 0을 Ctrl+D로 복사 → 'Level 1'

Animator에서 Level 1 상태의 Motion에 연결



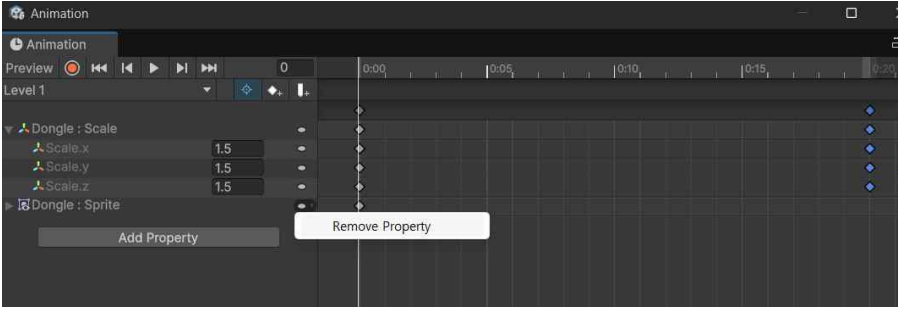


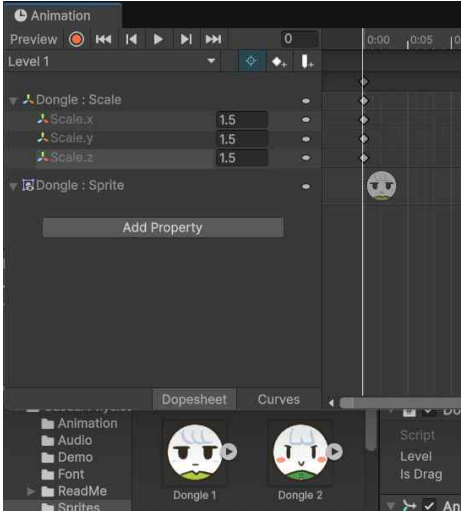


Level 1 애니메이션

마지막 키프레임 Scale 1 → 1.5, Sprite는 Dongle 1로

Level 2부터는 이미 세팅 완료! (예: Level 2는 최대 크기 2)




Animation
Preview
0
Level 1
0:00
0:05
Dongle : Scale
Scale.x
1.5
Scale.y
1.5
Scale.z
1.5
Dongle : Sprite
Add Property
Dopesheet
Curves
Animation
Audio
Demo
Font
ReadMe
Sprites
Dongle 1
Dongle 2
Script
Level
Is Drag

스크립트에서 애니메이션 실행

Animator anim; 선언 → Awake()에서 anim = GetComponent<Animator>();

OnEnable(): 오브젝트가 생성 · 활성화될 때 자동 실행되는 함수

anim.SetInteger("Level", level); → int 파라미터라 SetInteger 사용

```
public class Dongle : MonoBehaviour
{
    public int level;
    public bool isDrag;
    Rigidbody2D rigid;
    Animator anim;

    void Awake()
    {
        rigid = GetComponent<Rigidbody2D>();
        anim = GetComponent<Animator>();
    }
```

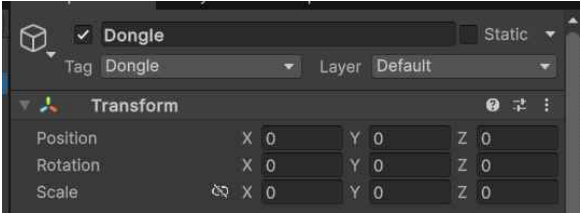
```
void OnEnable()
{
    anim.SetInteger("Level", level);
}
```

Prefab Scale 맞추고 테스트

애니메이션은 Scale 0에서 시작 → Prefab Scale도 1 → 0으로

저장 후 실행 → 쑥 커지며 나타나는 Dongle 확인!

그런데 레벨 0만 나와요... 다양한 레벨을 원해요!



랜덤 레벨 지정하기

GameManager의 NextDongle()에서 lastDongle.level 접근 (public이라 가능!)

lastDongle.level = Random.Range(0, 8);

최댓값은 포함 X → 0, 1, 2 ... 7까지 나와요

```
void NextDongle()
{
    Dongle newDongle = GetDongle();
    lastDongle = newDongle;
    lastDongle.level = Random.Range(1, 8);
    lastDongle.gameObject.SetActive(true);
    StartCoroutine(WaitNext());
}
```

활성화 순서 맞추기

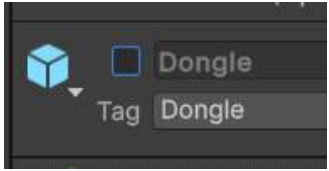
애니메이션은 OnEnable()에서 실행되죠

레벨 먼저 정하고 → lastDongle.gameObject.SetActive(true);

Prefab이 이미 켜져 있으면 의미 X → Prefab 비활성화(체크 해제)

실행하면 다양한 레벨의 Dongle 등장!

```
void NextDongle()
{
    Dongle newDongle = GetDongle();
    lastDongle = newDongle;
    lastDongle.level = Random.Range(1, 8);
    lastDongle.gameObject.SetActive(true);
    StartCoroutine(WaitNext());
}
```



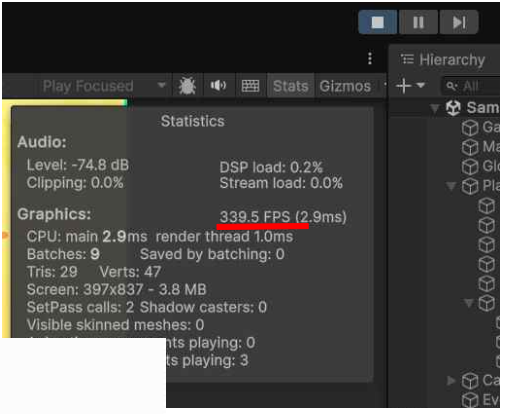
프레임 안정화

Game 창 위쪽 Stats → FPS가 불안정해요

일반적인 게임은 60프레임이 기준

GameManager Awake()에서
Application.targetFrameRate = 60;

어느 플랫폼에서나 60프레임으로 고정!



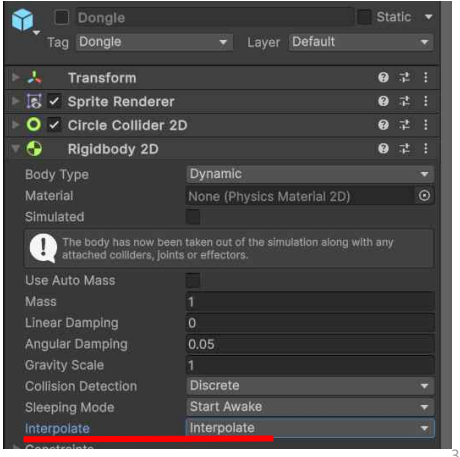

```
void Awake()
{
    Application.targetFrameRate = 60;
}
```

부드러운 낙하 (Interpolate)

Dongle Prefab → Rigidbody2D → Interpolate

None → Interpolate로 변경

Transform이 아닌 물리력으로 떨어지는 움직임을 부드럽게 보정!



다음 시간에는

GameManager로 다양한 레벨의 Dongle 생성 완성!

다음 시간엔 같은 Dongle끼리 합쳐지게 만들어 볼 거예요

다음 시간에 이어서 진행할게요 :)